

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 03 · LECTURE NOTES

Symmetric Encryption

Confidentiality, and the modes that provide it.

MODULE OUTCOMES

- Explain what a block cipher is and is not.
- Explain why ECB is unacceptable and what it reveals.
- Explain CTR and CBC and their requirements.
- Explain stream ciphers and their single fatal misuse.
- Choose a mode, and explain why you should not have to.

1 The block cipher

A BLOCK CIPHER IS A KEYED PERMUTATION ON FIXED-SIZE BLOCKS, AND NOTHING MORE

AES maps a 16-byte block to a 16-byte block, invertibly, under a key — and for each distinct key it is a different permutation of the 2^{128} possible blocks, selected from an astronomically large family.

The security goal is **indistinguishability from a random permutation**: an adversary permitted to encrypt and decrypt blocks of their choosing cannot tell AES under an unknown key from a permutation chosen uniformly at random — which is Module 01 §3's framing applied to the primitive.

But a single block is not a message. Encrypting anything longer requires a *mode of operation*, and the mode is where the real design decisions and essentially all of the real failures live — AES itself has no practical break after twenty-five years of attention.

So AES is not an encryption scheme. It is a component of one, and this distinction is the entire content of this module — which is why 'we use AES-256' in a system specification conveys almost no information about the system's security.

2 ECB, and what it teaches

ENCRYPTING EACH BLOCK INDEPENDENTLY LEAKS EVERY REPETITION

ECB mode applies the block cipher to each plaintext block separately and independently — so **identical plaintext blocks produce identical ciphertext blocks**, always, visibly, and across the whole message.

Which means structure survives encryption intact. The classic demonstration encrypts a simple bitmap in ECB mode and the image remains plainly recognisable — edges, shapes, and text all legible — because the repetition pattern of the plaintext is preserved exactly in the ciphertext.

And it fails the Module 01 §3 definition immediately. An adversary submits one plaintext consisting of two identical blocks and one consisting of two different blocks, and distinguishes the ciphertexts with certainty — advantage 1, which is as badly as a scheme can fail.

So ECB is unacceptable for any message longer than a single block — and **it is worth knowing precisely why, because 'AES' in a specification does not tell you the mode**, and ECB remains the default in more than one widely used library's lowest-level interface.

ECB block by block, independently.
Needs: nothing. Leaks: all repetition. Never use.

CBC each plaintext block XORed with the previous ciphertext block before encryption.
Needs: an unpredictable IV, and padding.
Hazards: padding oracles; IV reuse leaks the first-block relationship; not parallelisable for encryption.

CTR encrypt a counter, XOR the result with the plaintext. Turns a block cipher into a stream cipher.
Needs: a nonce that NEVER repeats per key.
No padding. Parallelisable. Hazard: nonce reuse is catastrophic (Module 11).

AND NONE OF THESE AUTHENTICATE ANYTHING. All three provide confidentiality only, which is why Module 04 exists and why you should be using AEAD instead.

The last paragraph is the point of the slide. Every mode listed is a confidentiality-only construction, and students who learn the modes without that caveat go on to build unauthenticated systems — which is the failure Module 01 §1's callout describes and Module 04 exists to prevent. **CBC's padding oracle hazard is worth noting specifically:** a system that reports padding errors distinguishably from other errors can have its plaintext recovered without the key, which is a decryption failure arising entirely from error handling.

3 Stream ciphers

$$c = m \oplus \text{KS}(k, n) \quad \text{so} \quad c_1 \oplus c_2 = m_1 \oplus m_2 \quad \text{if } n \text{ is reused}$$

- **KS(k, n)** — a keystream of arbitrary length generated deterministically from the key and the nonce.
- $\oplus$ — bitwise XOR, which is the entirety of the encryption operation. The cipher's work is producing the keystream.
- $c_1 \oplus c_2$ — what an adversary computes directly from two ciphertexts if the nonce repeats under the same key.
- **The consequence — the keystream cancels and the key disappears from the equation entirely**, leaving the adversary with two plaintexts XORed together. **That is a classical two-time-pad problem and it is solvable** for any plaintexts with structure, which is all real plaintexts. **This is the single most important equation in the course**, and Module 11 §2 returns to it.

CHACHA20 IS THE STREAM CIPHER TO KNOW

It is fast in software without hardware acceleration — which matters on mobile, embedded, and older targets where AES instructions are absent and software AES is both slow and hard to make constant-time.

It is structurally simple: additions, rotations, and XORs on a 512-bit state, with no lookup tables — which makes a constant-time implementation straightforward rather than delicate, because there are no data-dependent memory accesses to leak through the cache (Module 11 §1).

And it pairs with the Poly1305 authenticator into an AEAD construction (Module 04) **that is the default choice wherever AES-GCM's hardware support is unavailable** — and is specified in TLS 1.3 for exactly that reason (Module 08).

But it inherits the fatal requirement of every stream cipher: the nonce must never repeat for a given key — which is why the libsodium interface generates the nonce itself and simply does not offer you the option of supplying one. **That API decision is Module 01 §4's rule in concrete form.**

4 Choosing, and why you should not have to

- **Use an AEAD construction** (Module 04) — **AES-GCM, ChaCha20-Poly1305, or AES-GCM-SIV**, and not any confidentiality-only mode from §2 regardless of how carefully you intend to add authentication yourself.
- **Use the library's single-call interface**, which handles the nonce generation, the authentication tag, and the output encoding together as one operation that cannot be half-performed.
- **AES where hardware acceleration exists** — nearly every current server and desktop processor — **and ChaCha20 where it does not. This is the only legitimate choice in the module**, and most libraries will make it for you.
- **And if you find yourself selecting a mode by name, you have gone wrong somewhere** — which is Module 01 §4's IV test in a slightly different form, and it applies equally.

- **This module exists so that you can read an existing system and assess it**, not so that you can assemble a new one. **The honest purpose is diagnostic**: you will inherit systems documented as 'AES-encrypted', and you now know that sentence is incomplete in at least three respects — mode, authentication, and key management.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boneh & Shoup, chapters 3 through 5 (free)	§1 through §3 rigorously — including the proofs that CTR and CBC achieve the Module 01 §3 definition given their requirements.
Aumasson — Serious Cryptography, chapters 4 and 5	The modes and stream ciphers at engineering level , with the practical hazards foregrounded.
RFC 8439 — ChaCha20 and Poly1305 (free)	§3's specification , and short enough to implement from — which is one of Project 1's options.
The ECB penguin, and why it persists (free)	§2's demonstration, with the nuance about what the image does and does not prove.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Encrypt a bitmap in ECB mode** and look at the result.
- 2 **Construct the two-block distinguishing attack** on ECB and state the advantage you achieve.
- 3 **Encrypt the same plaintext twice in CBC with the same IV** and compare the ciphertexts.
- 4 **Do the same with CTR and the same nonce**, then XOR the two ciphertexts together.
- 5 **Recover both plaintexts** from that XOR, given that one is English text.
- 6 **Write out why the key vanished** from the equation.
- 7 **Time software AES against ChaCha20** on a machine without AES instructions, if you have one.
- 8 **Find a library whose lowest-level API defaults to ECB**, and read its documentation's warning.
- 9 **Take one system you did not build** and determine its actual mode and whether it authenticates.
- 10 **Rewrite one confidentiality-only use** as an AEAD call.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What exactly is a block cipher, and what is its security goal?
- 2 Why is AES not an encryption scheme?
- 3 Why does ECB leave a bitmap recognisable?
- 4 Give the two-block attack and the advantage it achieves.
- 5 What does CBC need, and what are its three hazards?
- 6 What does CTR need, and what does it not need?
- 7 What do all three modes fail to provide?
- 8 Write the stream cipher equation and say what happens on nonce reuse.
- 9 Why did ChaCha20 displace the alternatives, and what does it inherit?
- 10 What should you actually use, and what is this module really for?